LazyCode: Compiling Interactive Intent into Batch Execution

A Query-Optimizer Architecture, Its Economics, and the Limits of Overnight Coding Agents

Gurunath Lunkupali Venugopal*

August 2026

Abstract

Interactive coding agents are built on an unexamined default: every model call is synchronous, priced at the realtime rate, with a person nominally waiting. Yet the work these agents do best — backlog burndown, migrations, test coverage, mechanical refactors — has no person waiting, and every major provider sells a second product for exactly that shape: batch APIs with a 24-hour completion window at half the online price. LazyCode is a coding agent built on the batch tier: a realtime model *plans*, and the plan executes overnight as barrier-synchronized *waves* of self-sufficient batch calls. We present its architecture as a deliberate transplant of database query processing — a typed operator algebra as the logical plan, rewrite rules as the optimizer, waves as the physical plan, and an event-sourced log as the transaction machinery — and we are precise about which parts of that transplant are load-bearing code today and which are architecture awaiting weight. We derive the economics condition governing the whole enterprise: against a prompt-cached interactive baseline, batch execution wins on input tokens only when batch rounds R and interactive turns T satisfy $R < T/5$ — a condition that softens to near-parity for *wide* waves once batch and prompt-cache discounts stack, shifting the architectural burden from depth-collapse to width-supply. We report the strongest-implemented property of the system (end-to-end crash safety across the three durability windows of a batch submission) and its honest gaps. Finally, we analyze whether this architecture should generalize into agentic frameworks (Pydantic AI with durable-execution backends, NVIDIA's NeMo Agent Toolkit): we give a four-part criterion — snapshottable environment, free deterministic oracle, width over depth, positive laxity — for when plan-online/execute-on-batch pays, conclude that the *wave executor* deserves a framework-agnostic library while the *planner and optimizer* are coding-domain artifacts that should not be ported, and identify the single upstream protocol change (deferred *model* requests, the analogue of Pydantic AI's deferred tools) that currently blocks every framework from the batch tier.

1 Introduction

Every mainstream coding agent — Claude Code, Codex, and their descendants — shares one interaction contract: a loop of synchronous model calls, each awaited by a harness, each priced at the realtime rate. The contract is right when a person is watching. It is an unexamined default when nobody is: the overnight backlog task, the 400-file mechanical migration, the test-coverage push that nobody wants to babysit. For that work, latency tolerance is measured in hours, and the providers already sell a product priced for it — OpenAI Batches and Anthropic Message Batches both offer a 24-hour completion window at 50% of the online price.

The catch is structural, not financial. A batch call cannot ask a follow-up question, cannot read the result of the previous call in the same submission, and may take hours to return. An agent loop is the opposite shape:

*Code: `github.com/rajagurunath/lazycode`. Companion serving-side paper: Tidal (`github.com/rajagurunath/tidal`).

deeply sequential, incrementally contextualized, interactively repaired. Moving agentic work to the batch tier therefore requires *compiling* it: transforming a deep interactive loop into a shallow, wide, self-sufficient plan before the first batch call is made.

LazyCode is that compiler, and this paper is its story — told with database-systems vocabulary because the mapping is the design, not decoration. A realtime model emits a *logical plan*: a typed DAG over an eight-operator algebra of engineering work. An *optimizer* rewrites it. A *physical planner* lowers it onto provider batch APIs as barrier-synchronized waves, grouped like batched row processing. An event-sourced log makes the whole execution crash-safe and exactly-once in effect, the way a write-ahead log makes a database recoverable. Data systems spent forty years moving from batch to streaming; agent systems, born streaming, have never asked what moving the other way would buy.

What it buys is governed by one inequality (Section 3): the honest baseline is not an uncached agent but a prompt-cached one paying $0.1\times$ for repeated context, so batch's flat $0.5\times$ wins on input tokens only when the plan collapses T interactive turns into $R < T/5$ batch rounds. Output tokens — uncacheable on either side — are an unconditional ~50% win. This single condition unifies the latency mechanism and the cost mechanism: the shallow-wide DAG is simultaneously what makes 24-hour latency tolerable *and* what makes the economics beat a well-engineered interactive baseline. We further show (Section 3.1) that because batch and prompt-cache discounts stack on today's APIs, the condition is width-dependent: a wave of N prefix-sharing calls needs only $R \cdot (0.05 + 0.95/N) < 0.1\,T$, which for wide waves relaxes toward $R < 2T$ — moving the design burden from *manufacturing depth-collapse* to *supplying width*, with consequences for generalization that Section 8 develops.

We make four contributions:

1. **An architecture** (Section 4): the query-processing transplant in full — operator algebra, rewrite rules with their database analogues, content-addressed wave formation, and a three-window crash-safety protocol for paid third-party submissions — with an implementation-truth account that separates shipped machinery from declared vocabulary (Section 7).

2. **An economics analysis** (Section 3): the $R < T/5$ derivation, its width-dependent refinement under discount stacking, and the honest confrontation with `service_tier: flex` — OpenAI's zero-architecture route to the same discount — which any batch-tier argument must now name as its baseline.

3. **The self-sufficiency discipline** (Section 5): how interactive intent becomes deferred execution — front-loaded context harvest under byte budgets, contract-constrained outputs, and the *assumption ledger*, which converts "blocked on a human" into "decided and flagged for morning review."

4. **A generalization verdict** (Section 8): a four-part criterion for when this architecture pays, a component-by-component transfer analysis into Pydantic AI + DBOS and NVIDIA's NeMo Agent Toolkit, the identification of deferred model requests as the missing upstream seam, and a concrete recommendation — build the wave executor as a library; do not port the planner.

Throughout, we distinguish three epistemic layers explicitly: what is *implemented and tested* (16,979 LOC, 368 tests), what is *designed with plumbing in place*, and what is *vocabulary only*. Research prototypes usually blur these; a paper about honest economics should not.

2 Background: the two products and the honest baseline

Batch APIs. Both major providers accept a file (OpenAI) or list (Anthropic) of independent requests, promise completion within 24 hours, and charge half the synchronous price. Results arrive as a set kept

for about a month. Three properties shape everything downstream: items are *independent* (no item can read another's output), *non-interactive* (no mid-flight clarification), and *opaquely scheduled* (providers publish no latency percentiles; most items complete in under an hour, but the tail is unbounded up to expiry). Neither provider supports per-item cancellation — a fact that reshaped our hedging design (Section 6).

The honest interactive baseline. A naive comparison prices the interactive agent at list rates and reports 60–85% savings. That baseline is a strawman: production agents use prompt caching, under which repeated context is billed at 0.1× base on cache reads. Our first design document made exactly this error; an adversarial review caught it, and the corrected analysis (Section 3) is one of this paper's contributions. The strawman survives in most published comparisons of batch pricing.

The newer complication. Since 2026, the choice is no longer binary. OpenAI's `service_tier: flex` charges batch rates on ordinary synchronous calls — slower, capacity-contingent (429 on shortage, unbilled), but requiring *zero* architectural change. And Anthropic's batch worker now runs the *server-side* agentic loop (web search, code execution, MCP connectors) with more iterations per turn than the synchronous path, plus batch-exclusive 300k-token outputs. The batch tier is thus no longer "the same API, slower and cheaper": it is architecturally different in both directions, and Section 8 weighs both.

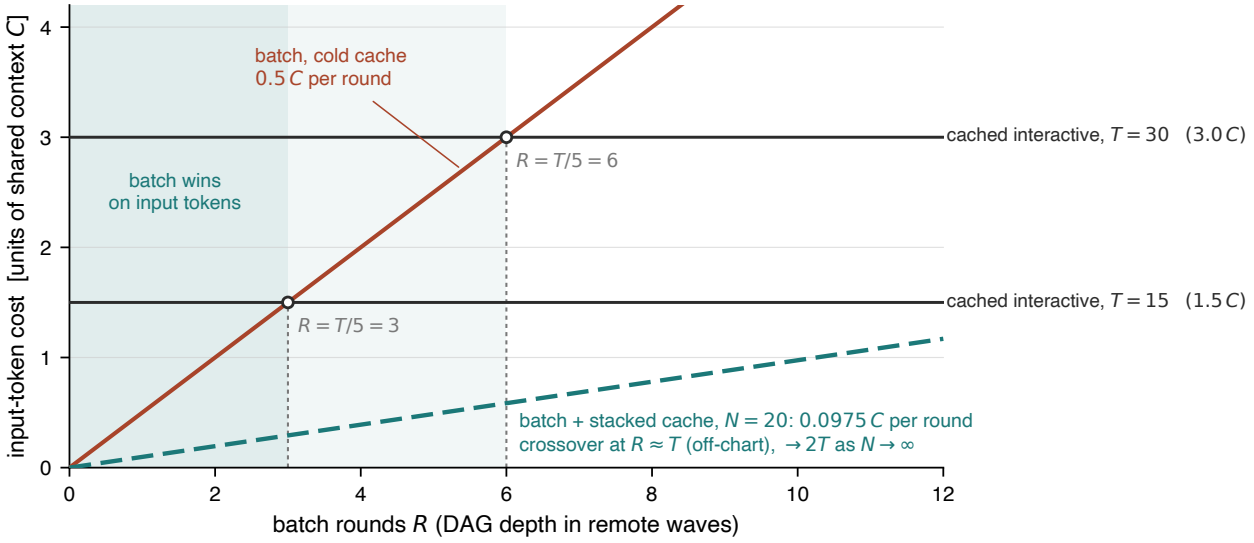


Figure 1: **The economics condition.** Input-token cost of completing one task, in units of its shared context C, as a function of batch rounds R. A prompt-cached interactive agent pays $\approx 0.1\,C$ per turn (horizontal lines for T=15 and T=30 turns). Uncached batch pays $0.5\,C$ per round: it beats the cached baseline only left of the R=$T/5$ crossovers. With batch and cache discounts stacked across a wave of N prefix-sharing items (dashed, N=20), the slope drops toward $0.05\,C$ and the crossover moves to $R \approx 2T$: width substitutes for depth-collapse. Output tokens (not shown) favor batch unconditionally at ~50%.

3 The economics condition

Let a task require shared context C (repository slices, house rules, instructions). An interactive agent across T turns re-reads that context every turn; with prompt caching, the input bill is approximately $0.1\,C\,T$ (cache

writes and the uncached first turn add small constants). LazyCode front-loads the same context into each of R batch rounds at the batch rate: $0.5\,C\,R$, assuming cold caches across waves. Batch wins on input iff

$$0.5\,C\,R \;<\; 0.1\,C\,T \quad\Longleftrightarrow\quad \boxed{R < T/5}.$$

A 30-turn interactive task compiled to 2–3 batch rounds costs 1.0–1.5 C against the cached agent's 3.0 C; the same task compiled poorly, to 8 rounds, loses. Output tokens are never cacheable, so batch's 50% discount on them is unconditional — and agentic coding is output-heavy. The blended saving is therefore "~50% on output plus a conditional input win": 30–70% against a well-cached baseline, and the widely-quoted 60–85% only against an uncached one. The design's own kill condition follows: if realized `rounds_per_node` degrades toward interactive depth, the input economics invert.

3.1 Width changes the condition

The derivation above assumes batch calls never hit caches. On current APIs that assumption is wrong in the favorable direction: batch and prompt-cache discounts *stack*, and Anthropic explicitly recommends the 1-hour cache TTL for batch workloads. Within one wave, N items sharing a hoisted prefix C pay approximately $C \cdot 0.5 \cdot 2$ once (cache write at the batch rate) and $C \cdot 0.5 \cdot 0.1$ for the remaining $N-1$ reads:

$$\text{per-item input} \;\approx\; C\,R\left(0.05 + \tfrac{0.95}{N}\right) \quad\Longrightarrow\quad \text{batch wins} \;\Longleftrightarrow\; R\left(0.05 + \tfrac{0.95}{N}\right) < 0.1\,T.$$

At N=1 this is $R < T/5$; at N=20, $R \lesssim T$; as $N \to \infty$, $R < 2T$. Two consequences. First, *width is the real currency*: a coding agent must manufacture it by splitting one task along independence boundaries, but fleet workloads (evaluation sweeps, corpus processing) have width by construction — the key fact behind Section 8's verdict. Second, cache hit rates are best-effort (providers cite 30–98%); the honest statement is the sensitivity range, not a point estimate, and within-wave hit rate belongs in the benchmark suite as a first-class metric.

3.2 The flex-tier confrontation

OpenAI's flex tier delivers batch pricing on synchronous calls: one string, no architecture, minutes of extra latency instead of hours. Any defense of the batch-compilation architecture must therefore rest on what flex does *not* provide: the 24-hour window that buys schedulable width (waves timed for cache stacking), the batch-exclusive server-side loop depth and 300k outputs, cross-run pooling, and — on the self-hosted side — the co-serving economics of the companion system (Tidal), where the batch tier monetizes idle capacity a flex-style tier would still consume at peak. For pure per-call discount on an interactive loop, flex dominates; this paper's architecture earns its complexity only where those additional properties are the point.

4 Architecture

4.1 The logical plan: an algebra of engineering work

Eight typed operators form the planning vocabulary: Explore, Decompose, Generate, Edit, Verify, Judge, Reduce, Gate. Only Generate and Edit carry a `context_spec` (what to harvest) and an `output_contract` (what shape must come back) — they are the operators that spend money and mutate the tree. The plan is a Pydantic-validated DAG: unique ids, resolvable dependencies (including `"node.*"` fan-out patterns), acyclicity by iterative DFS. Critically, the planner cannot drift from this schema, because the planning prompt's tool definition *is* `Plan.model_json_schema()` and the algebra documentation shown to the model

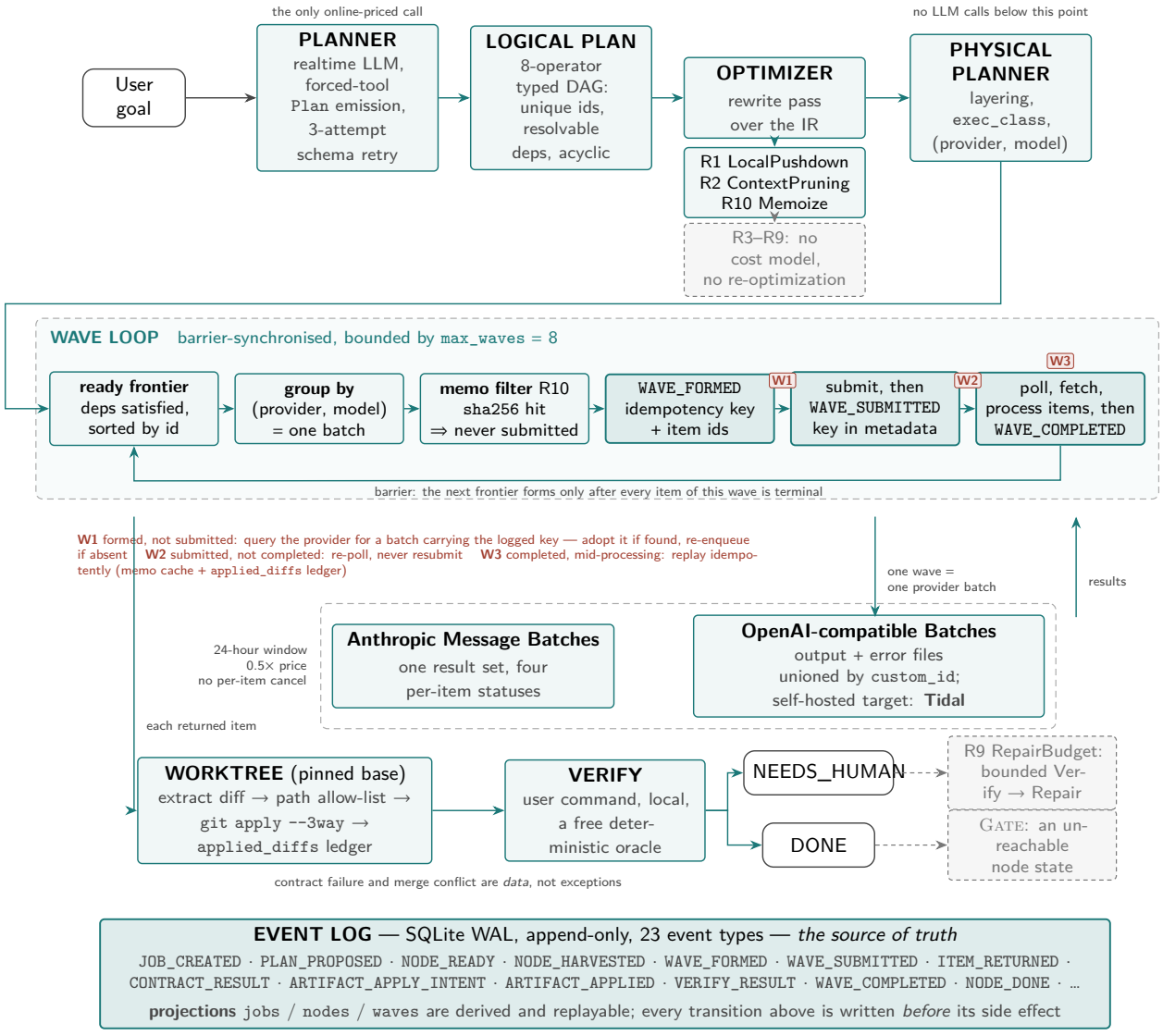


Figure 2: **The LazyCode pipeline.** Solid/accent components are shipped code; dimmed components are designed with plumbing but not yet live (Section 7). The event log is the source of truth; `jobs`/`nodes`/`waves` tables are replayable projections. W1–W3 mark the three crash-safety windows of Section 4.4.

is generated from the model fields themselves; a validation error is fed back verbatim for up to three repair attempts.

The DAG-shape thesis is executable prompt text, quoted in full because it is the paper's mechanism in one sentence:

> *"Structure the work as a shallow, WIDE DAG (few dependency layers, much fan-out) because wall-clock is proportional to DAG depth: split independent work (per-file, per-module) into sibling nodes at the same layer instead of chaining it."*

4.2 The optimizer: rules with database analogues

Table 1 lists the ten rewrite rules with their database lineage. Two ship today; the table's Status column is load-bearing (Section 7).

Table 1: Optimizer rules, their database analogues, and implementation status. "Plumbed" = data model and call paths exist; the decision logic does not.

Rule	DB analogue	Status	Semantics
R1 LocalPushdown	predicate pushdown	**shipped**	EXPLORE answerable by local tooling (ripgrep, AST outline) executes free, no LLM round
R2 ContextPruning	projection pruning	**shipped**	nodes touching ≤2 literal files drop the repo map from their context
R3 FanoutWidening	join reordering	prompt-level	split coarse nodes along independence boundaries; today width comes from the planner prompt, not a rewrite
R4 PrefixFactoring	common subexpr. elim.	partial	shared context hoisted to a cache-hinted prefix block; per-call today, not per-wave
R5 ModelTiering	access-path selection	designed	cheapest model whose predicted verify-pass rate clears threshold; escalate on failure
R6 Vectorize	batched row processing	plumbed	pack k homogeneous tasks into one call; `call_items` maps 1 call to k nodes (k=1 today)
R7 Speculate	branch prediction	designed	submit enumerable continuations as siblings — restricted to *remote*-decider branches after review showed the local-decider case saves zero waves
R8 HedgeInsertion	hedged reads	designed	stall detection + realtime duplicate for critical-path items
R9 RepairBudget	—	designed	bounded Verify→Repair loops, then NEEDS_HUMAN
R10 Memoize	materialized views	**shipped**	`sha256(model, prompt, mode, sample_idx)` keys an exact-reuse cache; identical calls never re-bill

4.3 Wave formation and the barrier ruling

The scheduler runs a bounded loop over the *ready frontier*: nodes whose dependencies are all successful, sorted by id for deterministic apply order. Remote frontier nodes group by (provider, model); each group be-

comes exactly one provider batch — a *wave*. Waves are barriers: the whole frontier submits, the orchestrator polls to completion, results are processed, and only then does the next frontier form. The design review's most contested ruling chose barriers over rolling flushes, for two reasons that compound: the cost model stays literal (wall-clock ≈ depth × wave latency) and the acceptance test stays countable. Rolling execution is deferred, not rejected.

Wave identity is *content-addressed*: the idempotency key is a hash of the rendered items plus a flush ordinal recovered by scanning the durable log — not an in-memory counter — so a crashed-and-resumed orchestrator derives the same key and can adopt its own orphaned submission.

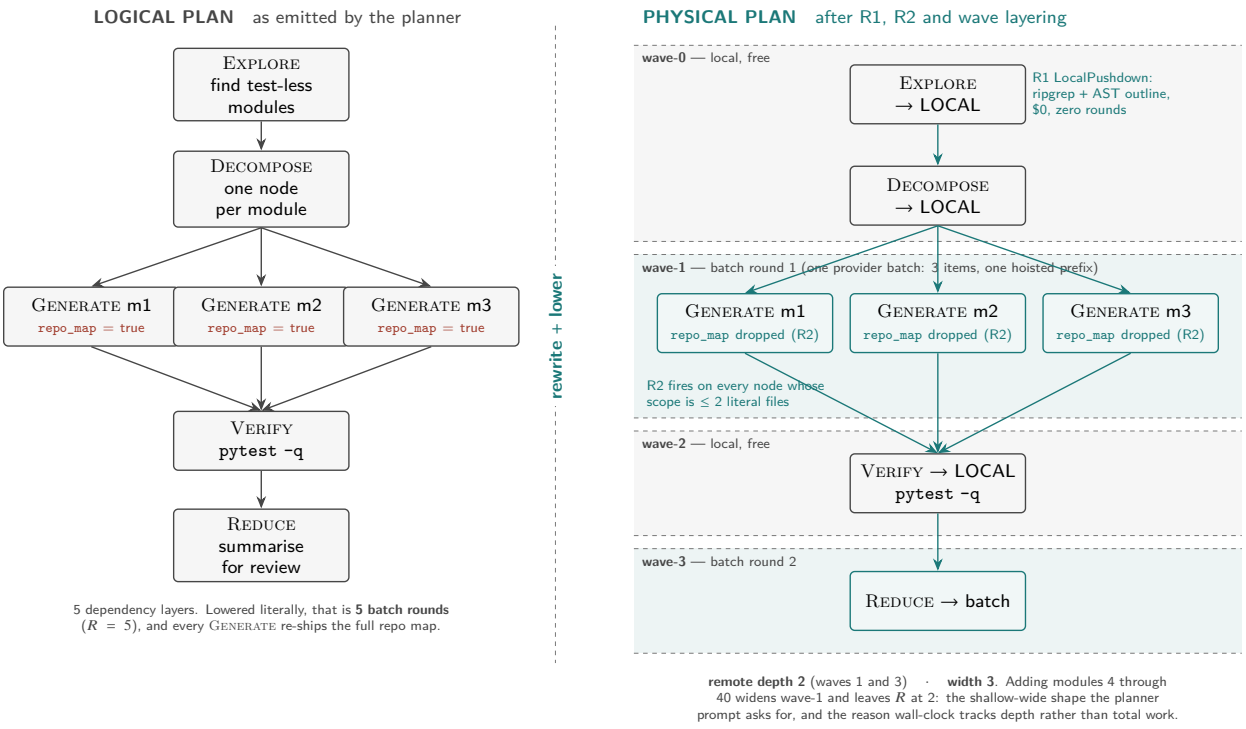


Figure 3: **A plan, compiled.** Left: the planner's logical DAG for "add tests for three modules." Right: after rewriting and lowering — the EXPLORE is pushed to free local execution (R1), trivially-scoped GENERATE nodes shed their repo map (R2), and layering yields two remote rounds regardless of module count: depth stays constant while width scales.

4.4 Crash safety: three windows, three recoveries

A batch submission is a paid, remote, irrevocable side effect, and the system treats it with WAL discipline. `WAVE_FORMED` — carrying the idempotency key and item ids — is durably logged *before* any provider call; `WAVE_SUBMITTED` after; `WAVE_COMPLETED` only after every returned item is fully processed. Each window has a distinct recovery: (W1) formed-not-submitted resolves by querying the provider for a batch bearing the logged key — found means adopt, absent means re-enqueue; (W2) submitted-not-completed re-polls and never resubmits; (W3) completed-mid-processing replays idempotently, because item results route through the memo cache and diff application checks a content-hash ledger before touching the worktree. Independent belts and braces: provider-side metadata, log-derived keys, the `applied_diffs` ledger, and a `UNIQUE(memo_key)` constraint on the spend cache. This is the strongest-implemented subsystem (Sec-

tion 7), and deliberately so: in an architecture whose entire premise is unattended execution, recovery is the product.

5 From interactive intent to self-sufficient calls

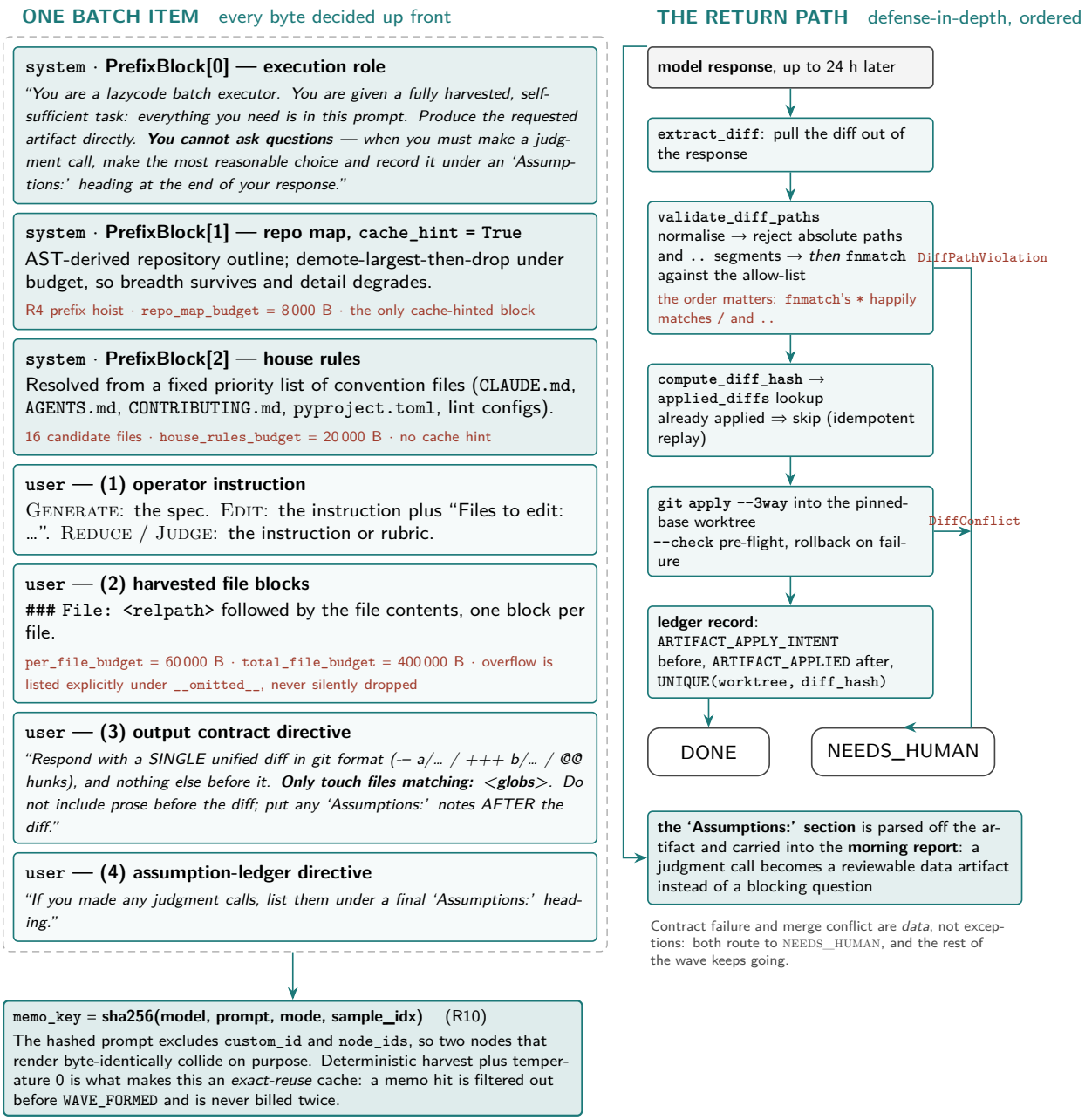


Figure 4: **Anatomy of a self-sufficient call.** Every byte the model will see is decided before submission, under explicit budgets; the output contract constrains the return shape; judgment calls come back as a structured *Assumptions* section feeding the morning report rather than blocking execution.

Three mechanisms convert interactive intent into deferred execution.

Front-loaded harvest under byte budgets. Context is gathered *before* submission, deterministically: an AST-derived repository outline (demote-largest-then-drop under an 8 KB budget — breadth survives, detail degrades), the referenced files (60 KB each, 400 KB total, overflow explicitly listed as omitted), and house rules resolved from a fixed priority list of convention files. Determinism matters twice over: identical repo state yields byte-identical context, which is what makes the memo key (R10) an exact-reuse cache rather than a heuristic one.

Contracts on the way back. A GENERATE/EDIT node declares its output contract — today, a unified diff confined to a glob allow-list. Validation is defense-in-depth ordered: path normalization rejects absolute paths and traversal *before* glob matching (because `fnmatch`'s `*` happily matches `/` and `..`), then a three-way `git apply` against a pinned-base worktree, with rollback on conflict and a content-hash ledger making application idempotent. Contract failure and merge conflict both route to NEEDS_HUMAN — failure is data, not an exception.

The assumption ledger. The prompt contract states it plainly: *"You cannot ask questions — when you must make a judgment call, make the most reasonable choice and record it under an 'Assumptions:' heading."* The planner records its own scope-level judgment calls in the plan; executors append theirs after each artifact. What would be a blocking clarification in an interactive loop becomes a reviewable data artifact in the morning report. This is the interaction-model transplant in miniature — and, we will argue, the single most portable idea in the system (Section 8).

The tuning surface. The parameters that actually govern how intent decomposes today: the trivial-scope threshold for context pruning (2 files), the four harvest byte budgets, `max_waves` (the depth ceiling, default 8), the uniform (provider, model) assignment (which makes wave count equal DAG depth), sampling temperature 0 (which makes memoization sound), and the planner's schema-retry budget. A second tier — the cost slider λ, per-job budgets and deadlines, hedge timers, vectorize k — is parsed and stored but not yet consumed (Section 7); we list it as designed surface, not live control.

6 Provider reality

Three discoveries from building against real batch APIs, each of which reshaped the design. (1) *No per-item cancellation exists* on either provider — only whole-batch cancel. The original hedging design assumed it; hedging was rebuilt as duplicate-and-discard: a realtime copy races the stalled batch item, first result wins at the node level, the loser is billed and dropped. (2) *Idempotency must be provider-visible*: both adapters stamp the submission key into batch metadata (via `extra_body` on Anthropic, natively on OpenAI) so crash recovery can find orphaned batches by listing recent submissions — local bookkeeping alone cannot survive a crash between submit and log. (3) *The two wire protocols differ where it hurts*: Anthropic streams one result set with four per-item statuses; OpenAI returns two files (output and error) that must be unioned by `custom_id`, reports no per-item expired status (it is inferred from batch state and count remainders), and silently omits cancelled items. The OpenAI-compatible adapter is what lets LazyCode execute against *self-hosted* batch tiers — including Tidal, the companion system that co-serves batch work on the same GPUs as online traffic, closing the loop from client intent to self-owned capacity.

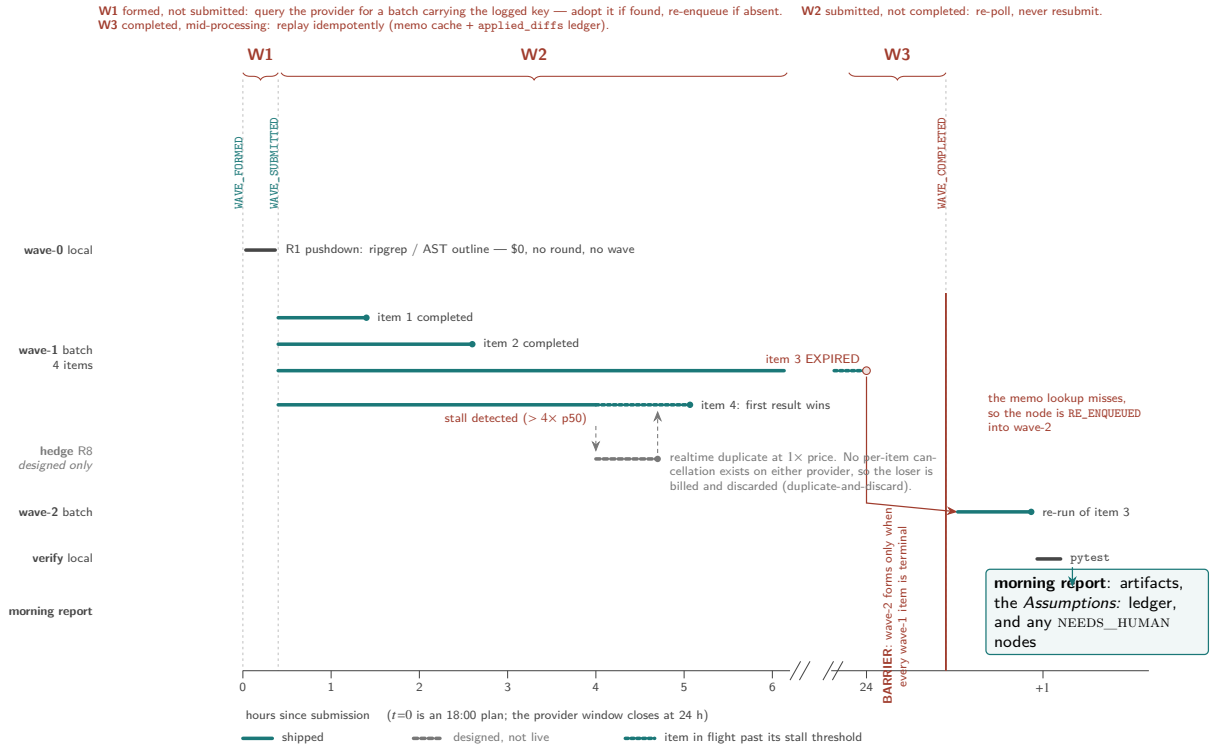


Figure 5: **Wave execution over a night.** Barriers separate waves; within a wave, items complete independently. The hedge path (dimmed: designed, not yet live) races a realtime duplicate against a stalled item under duplicate-and-discard semantics — the redesign forced by the absence of per-item cancellation. Expired items re-enter the next wave through the memo check. W1–W3 are the crash windows of Section 4.4.

7 Implementation truth

The system is 16,979 lines of Python (8,845 in the package, 7,281 in tests, 853 in the benchmark harness) with 368 tests. Three findings from auditing our own code, reported because papers rarely do and reviewers always should.

The optimizer is an architecture, not yet an optimizer. The shipped rewrite layer is 200 lines implementing two rules; R2's entire rewrite is one boolean flip under a hardcoded two-file threshold, and R1 trusts a planner-supplied flag rather than analyzing answerability. There is no cost model, no adaptive re-optimization at wave boundaries, and `Caps`-aware splitting is validated post-hoc by adapters rather than planned around. The query-engine *shape* — typed IR, rewrite pass, physical lowering, execution — is real and tested; the query-engine *intelligence* is the roadmap. Relatedly, wave count equals DAG depth today only because a single model makes (provider, model) grouping degenerate, and the statically-computed layer assignment is discarded in favor of dynamically formed, content-addressed waves.

The state machine is ~40% aspirational. Of 21 declared node states, 8 are unreachable (hedging, speculation, repair, gating, cancellation); of 23 event types, 2 are never emitted. We keep the full vocabulary in the schema deliberately — events are append-only and renaming is a migration — but the paper's figures dim what the runtime cannot reach.

Crash safety is the strongest subsystem. Every claim in Section 4.4 traces to tested code paths: the three windows each have a dedicated recovery with a dedicated test, including kill-9-mid-wave resume that adopts the orphaned provider batch rather than double-paying. Where the rest of the system is honest scaffolding, this part is load-bearing today — which is the right inversion for an unattended-execution product.

Evaluation status. A benchmark harness (three fixture tasks, a pricing model, and a Claude-Code-CLI baseline runner) ships; the only committed result is a mock-provider plumbing run. The economics of Section 3 are therefore *derived and reviewed*, not yet *measured* — the honest gap a reviewer should press on, and the immediate next step now that the companion self-hosted batch tier (Tidal, evaluated on GPU in its own paper) provides a meterable execution target: the same fixture tasks run end-to-end against self-hosted batch capacity with per-item token ledgers, alongside realized `rounds_per_node` and within-wave cache-hit measurements.

8 Should this generalize? Frameworks, and an honest verdict

LazyCode is a coding CLI. The obvious question — asked of us, and by us — is whether plan-online/execute-on-batch belongs in general agentic frameworks: Pydantic AI with its durable-execution backends, NVIDIA's NeMo Agent Toolkit, LangGraph and kin. The answer we defend: *partially, and not as an architecture*.

The substrate exists; the seam does not. Durable-execution integration is a solved problem: Pydantic AI ships co-maintained DBOS, Temporal, Prefect and Restate backends behind a public capability interface, with cost-budget gating built in; DBOS alone provides exactly-once steps, checkpointed multi-day sleeps, deduplication ids (LazyCode's idempotency key, ready-made) and rate-limited queues — and its old Postgres objection has expired (SQLite is now the default). What no framework has is a way to *defer a model request*: Pydantic AI's deferred-execution protocol exists only for tools; a model call is a leaf that must complete inside its activity, where a six-hour block violates every backend's durability contract (Temporal activities default to 60-second timeouts; DBOS steps retry from scratch). The single upstream change that unblocks the entire

ecosystem is a `DeferredModelRequests` analogue of the existing deferred-tools protocol — terminate the run at a model call, resume from persisted history plus results. The transport machinery already ships; the protocol extension is small; an open issue has waited since May 2025 for exactly this design.

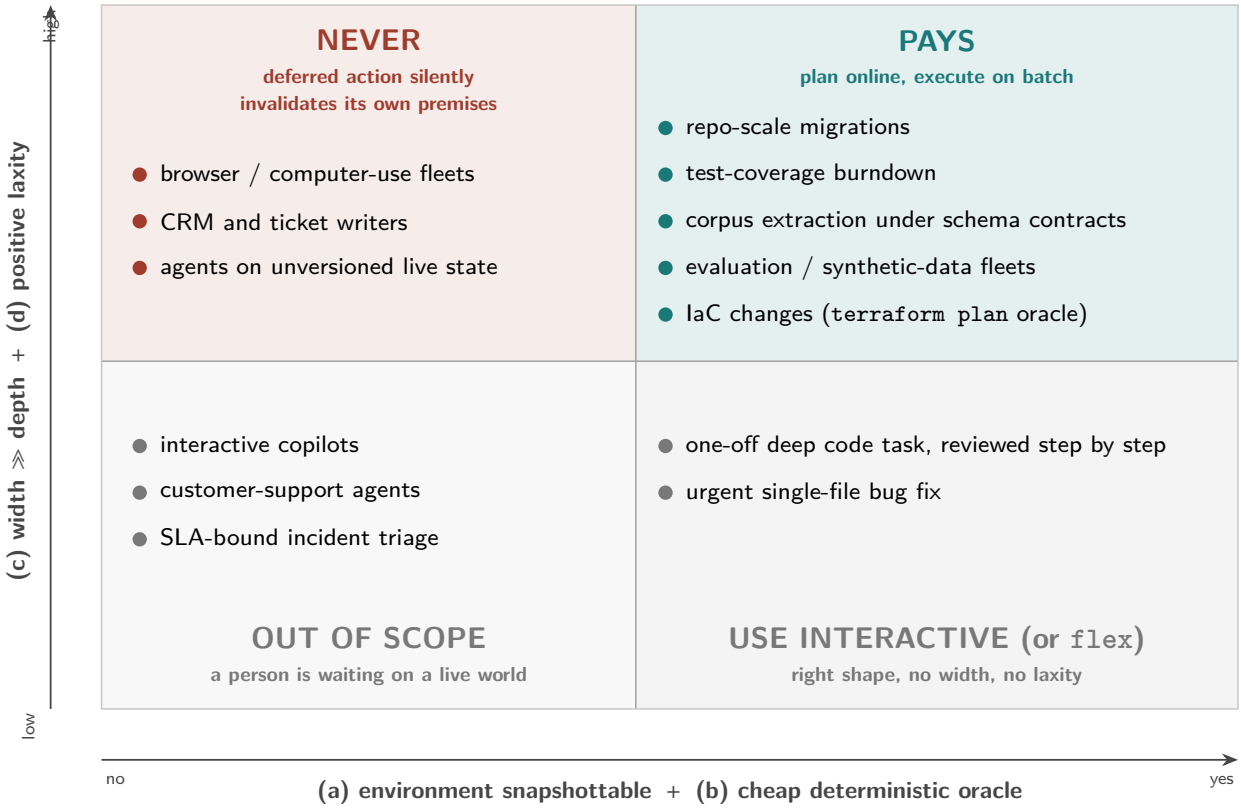
The negative control. NeMo Agent Toolkit — the most explicitly engineered framework in the field — has a genuinely strong profiler (including a common-prefix detector that independently rediscovers R4 from the serving side, and latency-tier routing hints into Dynamo) yet no durable suspension, no cost-tier concept, an empty cron stub, and no batch integration of any kind. The gap LazyCode fills is real across the ecosystem, not an artifact of coding CLIs. Meanwhile the only prior attempts at agent-side batch use are two 2026 fleet-pooling prototypes (a proxy that batches across a fleet of agents; a queue-flush router with per-call latency budgets) — both dispatcher-level, both stopping short of barriers, memoization, deadline machinery, and crash safety. Evaluation harnesses that *do* support batch APIs explicitly warn against using them for agentic tasks. The field has found the door; nobody has walked through it.

The criterion. The architecture pays exactly where a workload satisfies four conditions: *(a) the environment is snapshottable* — LazyCode pins a git commit and applies diffs into an immutable fork, so an 18:00 plan is still valid at 03:00; a CRM-writing or browser agent acts on a live world that moves during the wave, and deferred action against an unversioned environment silently invalidates its own premises; *(b) a cheap deterministic oracle exists* — pytest and compilers make VERIFY free and local; where the only verifier is an LLM judge, failure is discovered a wave late and the repair round erases the discount; *(c) width dominates depth* — structurally within the task, or by inter-run concurrency (Section 3.1); *(d) laxity is positive* — nobody is waiting. Coding is the canonical member of this class — versioned by git, oracled by test suites, fan-out-able by file — but it is a member, not the definition. Repo-scale migrations, IaC changes (`terraform plan` as oracle), corpus extraction under schema contracts, and evaluation/synthetic-data fleets all qualify. Interactive copilots, SLA-bound triage, and live-environment actuation never will; naming that boundary is a contribution, because a naive generalization into those classes would do real damage.

Transfer analysis and verdict. Component by component: the *wave executor* — barrier scheduling reinterpreted over N concurrent runs, memoization (R10 verbatim), prefix factoring (R4, now more valuable under discount stacking), vectorization plumbing (R6), laxity-driven {sync, flex, batch} routing with duplicate-and-discard hedging (R8), and the assumption ledger — transfers cleanly and is worth building as a ∼1–2k-line framework-agnostic library with thin DBOS/Temporal recipes and a Pydantic AI capability for cross-run pooling. The *planner, operator algebra, and code-analysis rules* (R1–R3, R9, the harvester) do not transfer: their content is ripgrep, per-file independence, and free test oracles — irreducibly coding. Our recommendation, in ascending order of leverage: build the library; ship the pooling capability; and above all land the deferred-model-request protocol upstream, because it is small, it is the true blocker, and it converts every durable-execution framework into a potential batch citizen at once.

9 Related work

Semantic operators and LLM query optimization. LOTUS, Palimpzest, Abacus, DocETL and Aryn establish cost-based optimization over declarative LLM pipelines; they own the operator-DAG-with-optimizer idea for *data* processing. LazyCode's distinction is the workload class — stateful, side-effecting, tool-using engineering work against a versioned environment with contract verification — and the batch-API execution substrate. *Model routing*: FrugalGPT and RouteLLM give the cost-tiering lineage for R5; the verify-pass-rate-gated variant is ours but unimplemented. *Agent-DAG serving*: Halo and Helium consolidate multi-agent queries



(a) LazyCode pins a git commit and applies diffs into an immutable fork, so an 18:00 plan is still valid at 03:00.
(b) `pytest` and compilers make VERIFY free and local; where the only verifier is an LLM judge, failure surfaces a wave late and the repair round erases the discount.
(c) width comes structurally from the task or from inter-run concurrency (§3.1); once batch and cache discounts stack, width is the currency.
(d) nobody is waiting.

Figure 6: **Where plan-online/execute-on-batch pays.** The four-part criterion as a map. The architecture's home turf is the upper right; the upper left is the danger zone where deferred execution silently invalidates its own premises.

into optimized DAGs with KV-sharing on *local* serving — the same instincts, the opposite deployment; sleep-time compute is adjacent (idle-time precomputation, not deferred pricing). *Coding agents*: SWE-agent's measured trajectory depths (≈13–15 turns) calibrate what T the economics condition must beat. *Batch tooling*: Curator, Ray Data LLM, and evaluation harnesses batch *independent* calls; none run an agentic loop through the batch tier, and the leading eval framework warns against trying. *Fleet pooling*: two 2026 prototypes mark independent convergence on inter-run batching, minus the systems machinery. *Serving side*: the companion Tidal paper supplies the self-hosted batch tier (deadline-contracted co-serving on vLLM) that closes the loop this paper's adapter opens. The unoccupied conjunction LazyCode claims: a barrier-scheduled, memoized, crash-safe, deadline-aware batch executor driving a stateful, contract-verified agent against a snapshottable environment.

10 Limitations

The economics are derived, not yet measured: no real-provider benchmark run exists, realized `rounds_per_node` is uninstrumented (the current counter can only report 1), and within-wave cache hit rates — on which Section 3.1 leans — are best-effort and unbooked. Provider latency tails are unpublished; the overnight guarantee is "most nights, with a priced fallback," not an SLA, and a hedge that fires re-buys realtime pricing for that item. The optimizer intelligence, hedging, repair, model tiering, vectorization $k>1$, and per-item cost accounting are designed but not live; the state machine carries their vocabulary ahead of their arrival. The flex tier undercuts the pure-discount motivation on one provider today. And the architecture's preconditions — git-snapshottable state, free oracles — bound its domain honestly: this is a compiler for a particular, valuable shape of work, not a universal agent runtime.

11 Conclusion

Data systems learned to go from batch to streaming; LazyCode runs the lesson in reverse for agents, and finds the pieces already lying around: a planning model that can emit typed DAGs, providers selling half-price deferred execution, database machinery for making it crash-safe, and — newly quantified here — discount stacking that rewards width more than heroic depth-collapse. The system's honest state is a strong skeleton with two shipped rewrite rules, bulletproof recovery, and unmeasured economics; its honest future is a measurement campaign against its companion self-hosted batch tier, and a small upstream protocol change that would let every agent framework defer a model call. The compiler metaphor sets the standard the project must ultimately meet: not that batch execution is possible — this paper demonstrates that — but that the compilation is profitable, case by case, with the receipts to show it.

Acknowledgments

The design benefited from adversarial multi-model review; several of this paper's central corrections (the cached-baseline economics, the vacant-speculation example, the width-dependent refinement) were surfaced by reviews that set out to break the claims rather than confirm them.

References

[1] L. Patel et al. LOTUS: Semantic Operators for Bulk LLM Processing. VLDB 2025. arXiv:2407.11418.

[2] C. Liu, M. Russo, M. Cafarella et al. Palimpzest: Optimizing AI-Powered Analytics. CIDR 2025. arXiv:2405.14696.

[3] Abacus: Cost-Based Optimization for Semantic Operator Systems. arXiv:2505.14661.

[4] S. Shankar et al. DocETL: Agentic Query Rewriting for Complex Document Processing. arXiv:2410.12189.

[5] L. Chen, M. Zaharia, J. Zou. FrugalGPT. arXiv:2305.05176.

[6] I. Ong et al. RouteLLM: Learning to Route LLMs. arXiv:2406.18665.

[7] Halo: Holistic Agent-DAG Optimization for LLM Serving. arXiv:2509.02121.

[8] Helium: DAG-Consolidated Serving for Agentic Workflows. arXiv:2603.16104.

[9] K. Lin et al. Sleep-time Compute. arXiv:2504.13171.

[10] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793.

[11] G. Lunkupali Venugopal. Tidal: An SLA-Contracted Batch Tier for LLM Serving Engines. 2026. github.com/rajagurunath/tidal.

[12] Pydantic AI: Durable Execution (DBOS, Temporal, Prefect, Restate). pydantic.dev/docs/ai, 2026.

[13] DBOS: Durable Workflows. docs.dbos.dev, 2026.

[14] NVIDIA NeMo Agent Toolkit (v1.8). github.com/NVIDIA/NeMo-Agent-Toolkit, 2026.

[15] UK AISI. Inspect AI: Batch Mode. inspect.aisi.org.uk/models-batch.html.

[16] Anthropic. Message Batches and Prompt Caching documentation. platform.claude.com/docs, 2026.

[17] OpenAI. Batch API, Flex Processing, and Webhooks documentation. developers.openai.com, 2026.

[18] E. Sandler. Batch API is terrible for one agent. It might be great for a fleet. Apr 2026. github.com/erans/batching-harness.

[19] llmfleet: latency-budget routing to batched Anthropic calls. github.com/MukundaKatta/llmfleet, 2026.

[20] Bespoke Labs. Curator. github.com/bespokelabsai/curator.

[21] G. Graefe. Volcano — An Extensible and Parallel Query Evaluation System. IEEE TKDE, 1994.

[22] Apache Spark: Adaptive Query Execution. spark.apache.org.